Digital Image Processing Lab (NCSH302)
B.Tech. 5th Sem (CSE Hons)

Session: Monsoon 2026-27
Date: 10.11.2026

Full Marks: 50

End Semester Examination

Time: 2 Hrs 30 Mins

Instruction:

i Solve **ONE** Question from Question No. 1 & 2. Question No. 3 is **compulsory**.

ii Save the source file (".py file") with a name as per the template given: "**Admission No_Question No.py**". For example, "24JE0161_Q1.py"

iii Create a folder with a name as per your "**Admission No._System No.**" and keep all source files inside of this folder.

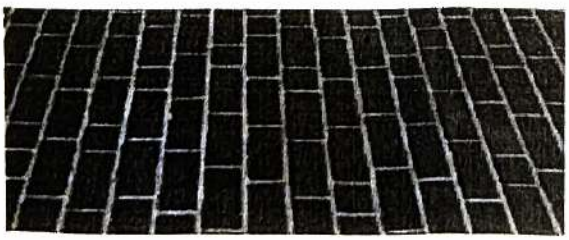
1. A smart construction site uses a drone to inspect brick walls for automated construction monitoring. Given an 8-bit grayscale image, brick.png (256×256 *pixels*), develop a Python program to segment and count visible bricks, perform lossless image compression, and generate a final survey image.

(a) Brick Segmentation and Counting: Smooth the input image using a normalized 5×5 Gaussian filter ($\sigma = 1$) with replicate padding. Compute the Sobel gradient magnitude, normalize it to [0, 255], and generate a binary edge map using a threshold of 80 (assign 1 to gradient magnitudes $\geq$ 80 and 0 otherwise). Apply morphological closing using a 3×3 square structuring element with zero padding. Perform 4-connected component labeling on non-edge pixels and retain components that do not touch the image boundary and contain at least one complete 3×3 window of non-edge pixels. Treat each retained component as a brick and calculate its centroid as $(\bar{x}, \bar{y}) = \left(\frac{\sum x}{A}, \frac{\sum y}{A}\right)$, where A is the number of pixels in the component and (x, y) are their coordinates. Number the bricks in ascending order of centroid y-coordinate, followed by x-coordinate. Display the binary edge map and numbered brick regions in grayscale and report the total number of detected bricks.

(b) Huffman Compression and Final Survey Image: Implement Huffman coding manually to encode all grayscale pixels of the original image into a single bit stream and decode them to reconstruct the image. Calculate entropy $H = -\sum p_i \log_2(p_i)$, average code length $L_{avg} = \frac{1}{MN}\sum p_i l_i$, coding efficiency $\eta = (H/L_{avg}) \times 100\%$, compressed size $S_c = MN \times L_{avg}$ bits, and compression ratio $CR = 8MN/S_c$, where p_i is the probability of gray level i, n_i is its frequency, l_i is its Huffman code length, and $M \times N$ is the image size. The compressed size represents the total number of encoded data bits, excluding codebook and file-header overhead. Verify exact reconstruction. Convert the reconstructed grayscale image into a three-channel RGB image, highlight detected brick pixels in blue $(0, 0, 255)$, and leave all remaining pixels unchanged. Display and save the final survey image as *final_image.png* and report the compression metrics and reconstruction result.

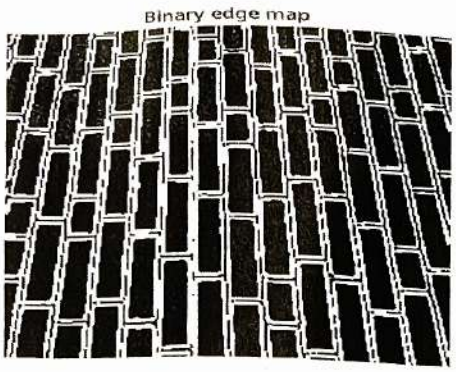
Instructions: *NumPy may be used for basic array operations and Matplotlib for visualization. All specified image-processing algorithms must be implemented manually.*

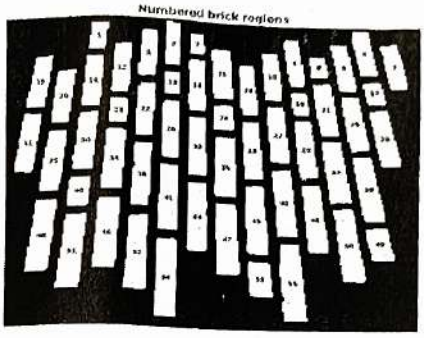
Sample Input:



Sample Output:

(a)

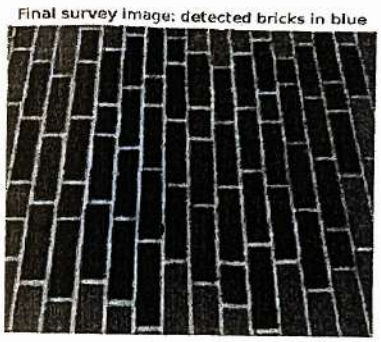




Detected bricks: 55

(b)

```
Entropy H: 5.4277 bits/pixel
Average code length: 5.4663
bits/pixel
Coding efficiency: 99.29%
Compressed size: 358239 bits
Compression ratio: 1.4635
Reconstruction exact: True
```



[Marks: 15 + 15]

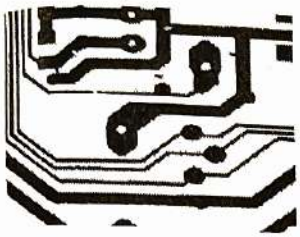
2. An automated inspection system detects manufacturing defects in printed circuit boards (PCBs) by comparing a test image with a defect-free reference image. Given two spatially aligned grayscale images, *PCB_Reference.jpg* and *PCB_Test.jpg*, develop an image processing system to detect, localize, and count missing and unwanted conductive regions. Assume that conductive regions appear darker than the PCB background.

   (a) Image Preprocessing and Segmentation: Resize both images to 512 × 512 pixels using bilinear interpolation. Apply a 3 × 3 median filter to the resized Test PCB, followed by a 3 × 3 averaging filter on both the resized Reference and median-restored Test images, using zero padding for both filtering operations. Perform iterative global thresholding independently on both smoothed images, initializing each threshold with the respective mean intensity and updating it as the average of the two group means until the absolute change is less than 0.5 (retain the current threshold if either group is empty). Generate binary images by assigning 1 to intensities ≤ threshold and 0 otherwise. Apply morphological opening using a 3 × 3 square structuring element with zero padding. Display the segmented and morphologically processed Test PCB images.

   (b) Defect Detection, Localization, and Counting: Compare the resulting morphologically processed binary images to identify missing conductive regions (Reference = 1, Test = 0) and unwanted conductive regions (Reference = 0, Test = 1). Apply 8-connected component labeling separately to both defect masks and discard components containing fewer than 10 pixels. Treat each retained component as one detected defect. Display the resized original Test PCB with all retained defect pixels highlighted in red, leaving the remaining pixels unchanged. Compute and report the total number of detected defects as the sum of retained components from both defect masks.

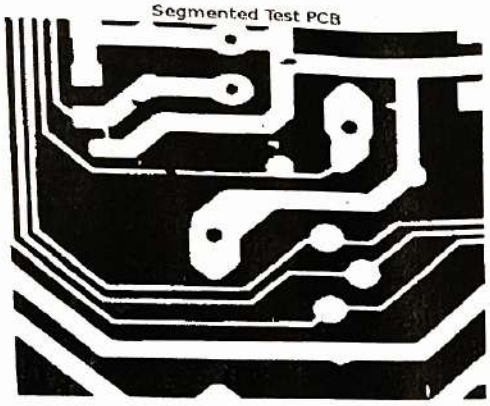
   Instructions: *Implement filtering, thresholding, morphological operations, and connected component labeling without using built-in image-processing functions.*

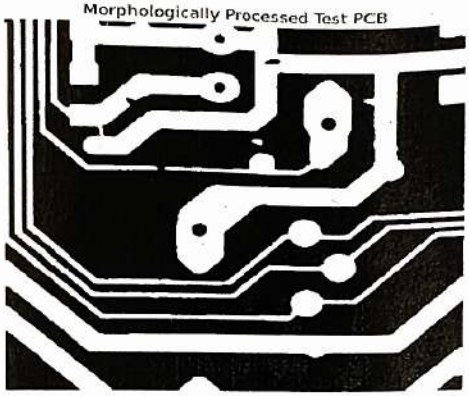
Sample Input:



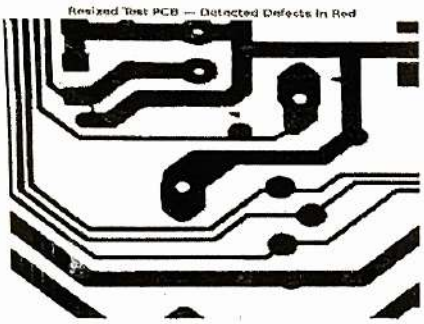
Sample Output:

(a)





(b)



Missing conductive-region components: 52
Unwanted conductive-region components: 61
Total detected defects: 113

[Marks: 15 + 15]

3. An 8-bit grayscale image, einstein.png, has low contrast because its gray levels occupy a narrow range within [0, 255]. Develop an image enhancement system using intensity transformations, histogram equalization, and spatial filtering.

   (a) Contrast Enhancement and Histogram Analysis: Read the grayscale image f and compute its minimum r_{min}, maximum r_{max}, mean, and median gray levels. Determine whether its histogram is positively or negatively skewed. Implement linear stretching, $s = round[255(r - r_{min})/(r_{max} - r_{min})]$, and logarithmic stretching, $s = round[255\, log(1 + r - r_{min})/log(1 + r_{max} - r_{min})]$, using separate 256-entry lookup tables, clipping input intensities to $[r_{min}, r_{max}]$. Assume $r_{max} > r_{min}$. Apply both transformations and report their mapped values for r = 100, 110, 120, and 140. Implement histogram equalization using $s = round[255 \times \Sigma_{(j=0\ to\ r)}\, n_j/(MN)]$ to obtain image h, where n_j is the number of pixels at gray level j and $M \times N$ is the image size. Display the original image, both stretched images, and the histogram-equalized image with their histograms. For each image, report its mean, standard deviation, number of occupied gray levels, and percentage of pixels in [0, 63], [64, 127], [128, 191], and [192, 255].

(b) Image Sharpening Using Laplacian Filtering: Sharpen the histogram-equalized image h by implementing a 2-D convolution function *convolve(image, kernel)* using replicate padding. Compute the Laplacian response L using the 4-neighbour kernel with center coefficient −4, four adjacent coefficients +1, and corner coefficients 0. Generate the sharpened image using s = clip(round(h − L), 0, 255), retaining signed intermediate values before clipping. Display the histogram-equalized image h, Laplacian response L scaled to [0, 255] for visualization, and the final sharpened image s.

Instructions: *Implement all enhancement, histogram equalization and convolution operations manually. NumPy may be used for basic array operations and Matplotlib for visualization.*

Sample Input:



Sample Output:

(a)

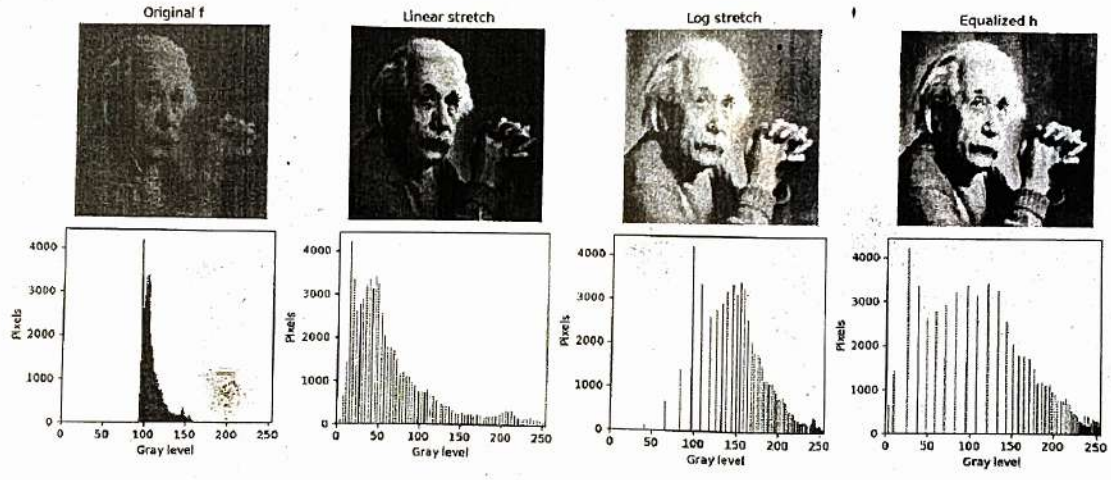


Image	Mean	Std	Levels	[0,63]	[64,127]	[128,191]	[192,255]
Original f	110.40	13.64	68	0.00%	88.68%	11.32%	0.00%
Linear stretch	66.20	51.95	68	62.21%	25.77%	7.02%	5.01%
Log stretch	161.30	42.22	68	0.18%	22.89%	51.97%	24.96%
Equalized h	131.82	71.58	60	23.07%	24.40%	27.57%	24.96%

(b)



[Marks: 15 + 5]